What happens when you remove layers from a small language model

A layer-ablation and logit-lens study of SmolLM2-135M

Wattanit Hotrakool · Emberly Oxygen project · November 2026

An exploratory note. Not peer reviewed. All numbers come from the experiments described here and can be reproduced with the code listed in the last section.

1 Summary

A transformer language model is a stack of identical-looking layers, and the model's answer is read off the output of the last one. We asked three questions of a small model (30 layers): can the answer be read earlier in the stack, does it matter *which* layers you drop, and when does the prediction become visible as it travels through the stack?

The findings, on one 135-million-parameter model and 12 short prompts:

- **Stopping early does not work without retraining.** Keeping the first 25 of 30 layers and reading the answer there agrees with the full model's next-token choice only about 50% of the time. Keeping the first 28 agrees 60% of the time.
- **The last layer cannot be used on its own.** Keeping the first 24 layers plus the last one is the worst result in the study: 0% agreement, on every prompt. The last layer only works when the layers just before it are intact.
- **Which layers you drop matters a great deal.** Removing a block of five layers from the middle (layers 12 to 18) keeps about 70 to 76% agreement, while removing five from the end (layers 24 to 28) gives 0%. Layer 0 is essential (2% without it) and one middle layer, layer 11, is unexpectedly important (43%).
- **The prediction appears late.** Reading the residual after each layer with the model's own output stage shows near-meaningless output (spaces and punctuation) until roughly layer 24, then a rapid emergence of the final prediction over the last six layers.
- **Dropping layers saves compute in proportion to the layers dropped, nothing more.** Five fewer layers costs about 13% less time per token, whichever five.

These are measurements of how much the output *relies* on each layer. They do not show what each layer computes. Section 8 lists what the study cannot say.

2 Background

This section explains only what is needed to read the rest. Readers who know transformers can skip to Section 3.

The model. A decoder-only language model reads a sequence of tokens (word pieces) and predicts the next one. Each token is first turned into a vector, its *embedding* (here 576 numbers). The vector then passes through a stack of *layers*. Each layer has two parts, an attention block (which lets the token look at earlier tokens) and an MLP block (a per-token feed-forward network). After the last layer, a final normalization and a multiplication by a matrix of size vocabulary times 576 turn the vector into one score per vocabulary entry (the *logits*); the highest score is the model's choice of next token.

The residual stream. Layers do not replace the vector. Each layer computes a correction and *adds* it to the vector it received. The running vector is called the residual stream, and every layer reads from and writes to the same stream. Two consequences matter here. First, a layer's input is whatever all earlier layers left in the stream, so layers are not independent stages of a pipeline. Second, removing a layer does not break the shape of the data: the stream simply lacks that layer's correction, which is why the experiments below are possible at all.

Tied embeddings. In this model the output matrix is the same matrix as the input embedding table (`tie_word_embeddings: true`). So reading a vector with the output stage asks "which token's embedding does this vector look most like?". This explains one result in Section 6.

The logit lens. The final norm and output matrix can be applied to the stream at any depth, not only after the last layer. This reads off what the model would predict if it stopped there. The technique is known as the logit lens. It is a crude view, because earlier layers were not trained to write in the form this output stage expects.

3 Setup

Model. SmolLM2-135M (a Llama-architecture model): 30 layers, hidden size 576, 9 attention heads with 3 key-value heads, feed-forward width 1536, vocabulary 49,152, tied embeddings. It is a base model, not tuned for chat.

Engine. All runs use Emberly Oxygen, a from-scratch educational inference engine in Rust that was checked against a reference implementation for agreement in its logits and intermediate values before this work. The experiments added two throw-away functions to the engine, which keep either the first `n` layers or an arbitrary list of layer indices and discard the rest. The key-value cache is rebuilt for the layers that remain, so nothing else changes. Layers keep their original weights; there is no retraining and no repair of the stream after a layer is removed.

Prompts. Twelve short prompts of different kinds: factual, arithmetic, code, narrative, a letter opening, a number sequence, a science sentence (listed in Section 10). For each prompt the full 30-layer model generated up to 32 tokens greedily (always taking the highest score). Prompt 4 (the code prompt) stopped after 29 tokens. This gives 381 scored tokens in total.

Metric: top-1 agreement. To score a modified model, we feed it the prompt followed by the *full model's own generated tokens* ("teacher forcing") in one pass, and at each generated position check whether the modified model's highest-scoring token equals the token the full model chose. Agreement is the fraction of matches. Teacher forcing matters: if we let the modified model generate freely, one early mistake changes everything after it, and the number would measure how errors compound rather than how well each prediction matches. A score of 100% means the modified model makes the same choice as the full model at every position. It does not measure whether the choice is correct or the text is good.

Free-running output. Separately, for the first experiment we also let each modified model generate on its own and quote the text, to give a feel for what the agreement numbers mean.

4 Experiment 1: stopping early

The simplest way to use fewer layers is to stop after layer `n` and read the answer with the normal output stage. Table 1 shows this for one prompt ("The quick brown fox jumps over the lazy dog, and then", 24 tokens). More outputs are in the sample-output appendix.

Layers kept	**Agreement**	**ms per token**	**Free-running text**
30 (full)	24/24	7.6	“ the dog jumps over the cat. The dog is a dog, and the cat is a cat.”
28	14/24	7.2	“ he p R high, and then he D a, and then he D a, and then he D a”
26	13/24	6.9	“ a / great, / a / a / a / a / a”
25	14/24	6.6	“ the / in the / in the / in the / in the / in the”
20	7/24	5.6	a stream of non-breaking spaces
10	3/24	3.6	“, , and, and, and, and, and, and”
1	4/24	1.8	“ is a”

Table 1: Stopping after the first n layers, one prompt. Slashes stand for line breaks in the generated text. Timing is one run on an Apple M4 Max and is indicative only.

Even two layers fewer (28) breaks the text. Over all twelve prompts (381 tokens), keeping the first 28 layers agrees on 59.6% of tokens and the first 25 on 49.9%. In the free-running text, the output typically collapses into repetition within a few tokens.

Cost. Time per token fell roughly in proportion to the layers removed (about 13% for five layers), as expected, because every layer does the same amount of work. The output stage costs the same at any depth, so it sets a floor: with one layer the engine still spent 1.8 ms per token.

5 Experiment 2: dropping layers from the middle

If the last few layers are what turns the stream into an answer, perhaps we can drop layers from the *middle* and keep the end. We tried several ways of keeping 25 layers, plus 28 and 27 (Table 2, 12 prompts, 381 tokens).

Layers kept	**Top-1 agreement**
All 30	100.0%
First 28	59.6%
First 25	49.9%
First 24 + last 1	**0.0%**
First 20 + last 5	34.6%
First 15 + last 10 (drops layers 15 to 19)	**70.9%**
First 10 + last 15	49.3%
Every sixth layer dropped (5, 11, 17, 23, 29)	32.5%

Table 2: Several ways of keeping a subset of layers. Layers are numbered from 0.

The result that stands out is “first 24 plus last 1”: the output is a stream of a single special token and agreement is exactly zero on all twelve prompts. It is far worse than just stopping at layer 25 (49.9%). Keeping the last layer without its immediate predecessors is worse than not having it. The best 25-layer arrangement here keeps the first 15 and the last 10.

5.1 Which layers matter: one at a time and in blocks

To see the structure, we removed each layer on its own (Figure 1) and every window of five consecutive layers (Figure 2). The bar shows top-1 agreement with the full model; longer is better.

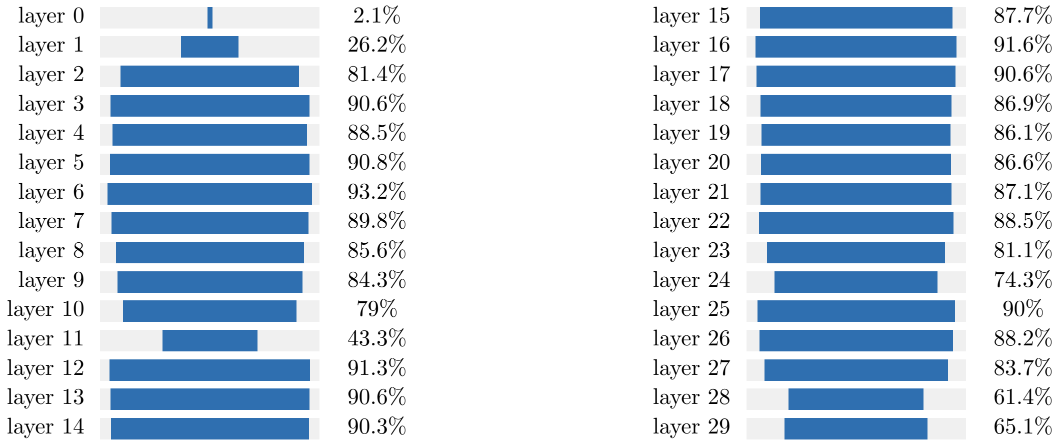

Figure 1: Agreement with the full model when one layer is removed (the other 29 kept).

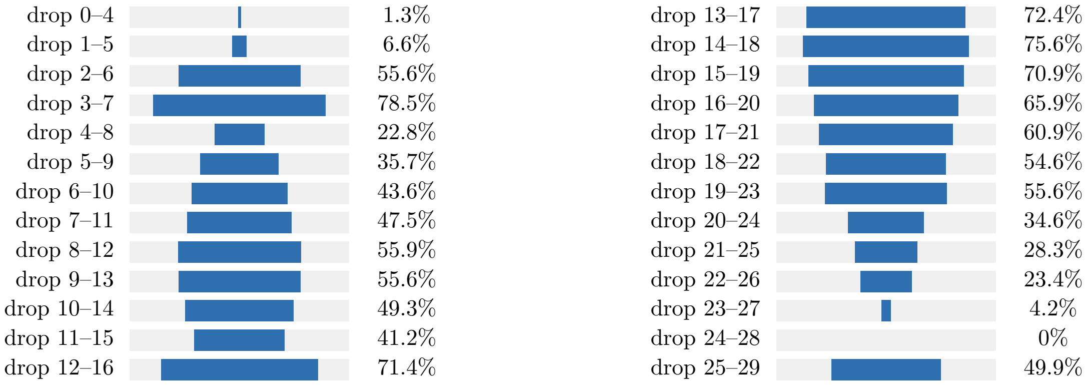

Figure 2: Agreement when five consecutive layers are removed (25 kept). The last bar, layers 25 to 29, is the same model as "first 25".

What the figures show:

- **Most single layers are individually dispensable.** Removing any one of layers 2 to 10 or 12 to 22 leaves 79 to 93% agreement.
- **Layer 0 is essential** (2.1% without it) and layer 1 nearly so (26.2%). Both sit directly on top of the token embeddings.
- **Layer 11 is an unexplained outlier.** Without it agreement is 43.3%, while layers 10 and 12 cost 21 and 9 points. We did not investigate why. A guess, not a finding, is that it writes something other layers rely on, for example a feature used as an attention anchor.
- **Layers 24, 28 and 29 cost more than their neighbours** (74.3%, 61.4%, 65.1%).
- **Removing five in a row from the middle is survivable, from the end it is not.** The best blocks are layers 3 to 7 (78.5%), 14 to 18 (75.6%), 13 to 17 (72.4%) and 12 to 16 (71.4%). The worst are 24 to 28 (0.0%) and 23 to 27 (4.2%).
- **The block results are not smooth.** Layers 3 to 7 give 78.5% but shifting the window by one, to 4 to 8, gives 22.8%. A single layer can matter more than where a block sits.

The 24-to-28 collapse matches the "first 24 plus last 1" result: both remove those five layers while keeping layer 29, and both give zero. Layer 29 expects the input left by layers 24 to 28.

6 Experiment 3: logit lens

The earlier experiments ask what the output relies on. This one asks *when the prediction becomes visible.* For each depth `n` from 0 to 30 we run the prompt through the first `n` layers and apply the normal output stage to the last prompt position, printing the three highest-probability tokens (probabilities from a softmax over the scores). Row 30 is the model's real output. Table 3 shows two prompts; the full listings for four prompts are in the last appendix.

After layer	Top 3 predictions (probability)	Full model’s top token
Prompt: “The capital of France is”		
30	“ the” 26%, “ Paris” 9%, “ located” 7%	26.2%
28	“ the” 72%, “ a” 9%, “ “ 6%	72.0%
26	“ “ 56%, “ the” 17%, “ a” 8%	17.1%
24	“ “ 43%, “,” 22%, “ the” 14%	14.4%
22	“ “ 89%, “,” 5%, “ a” 2%	0.6%
15	“ “ 78%, “,” 16%, “.” 3%	0.0%
5	“ “ 33%, “,” 20%, “.” 18%	0.0%
0	“ is” 100%	0.0%
Prompt: “def fibonacci(n): if n < 2: return n; return”		
30	“ fib” 95%, “ n” 1%, “ (“ 1%	94.8%
29	“ “ 45%, “ n” 23%, “ N” 2%	1.8%
28	“ “ 89%, “ (“ 4%, “-” 2%	0.0%
20	“ “ 78%, “ (“ 12%, “-” 7%	0.0%
0	“ return” 100%	0.0%

Table 3: Logit lens at selected depths. The last column is the probability the depth assigns to the token the full model chose.

Observations:

- **Zero layers repeats the last input token.** After no layers the stream is the token’s own embedding, and with tied embeddings the output stage returns that same token, always with probability near 1 (“ is”, “ then”, “ return”, “:”). This is a property of the tying, not a prediction.
- **Layers 1 to about 20 read as noise.** The top tokens are spaces, commas, periods and brackets, and the full model’s token has probability close to zero. These are the cheapest guesses for an output stage that cannot yet interpret the stream.
- **The prediction emerges in roughly the last six layers.** For the capital-of-France prompt, the final answer “ the” goes from 0.6% at layer 22 to 14% at 24, 17% at 26 and 72% at 28 (and then settles to 26%, as layer 29 and 30 move some probability to other tokens).
- **The last layer can do a large share of the work.** On the code prompt, layer 29 still predicts a space (45%) and the correct “ fib” is at 1.8%. The last layer takes it to 95%. This is consistent with Experiment 2: the last layer matters, but it depends on the layers before it.
- **Some predictions are simply easy.** After “Q: What is 12 plus 30? A:” a space is predicted with 41 to 99% probability from layer 1 onward, because a space after “A:” is a trivial format pattern; here the lens agrees with the final output at almost every depth. Which depth first shows the answer depends on how hard the prediction is.
- **The model barely knows the capital.** Even the full model gives “ Paris” only 9% and puts “ the” first. The study used a small base model; the capital-of-France result should not be read as a knowledge test.

7 Discussion

Are layers pipeline stages? Not cleanly. The residual stream is shared, so later layers can use what any earlier layer wrote, and each layer contributes a correction on top of what is there. This is why a single layer is often removable (Figure 1). But the layers are not interchangeable either: layer 0 and layer 1 are critical, one middle layer is unusually important, and layers 24 to 29 as a group are needed to produce the answer. A rough division of labour appears to exist (early layers set up the stream, the middle ones refine it, the last ones turn it into a prediction), but the experiments here show how much the output relies on each layer, not what each computes. From the literature, finer-grained roles belong to individual attention heads and MLP blocks rather than to whole layers; this study does not test that.

Why is the end so fragile? The output stage was trained against what the last layer writes, and the last layers were trained against what their predecessors write. Removing a late layer changes the input of everything after it, and the later layers have no way to adapt. Removing a middle layer changes the input of the layers after it too, but they are partly redundant with their neighbours, and the effect is smaller. This is our reading of the data, not something we tested directly.

Practical reading. If you wanted a cheaper model by deleting layers, the numbers say: delete from the middle, never from the last few layers, keep layers 0 and 1, and expect to need retraining or fine-tuning to recover quality. Even the best five-layer cut here lost about a quarter of the model's top-1 choices.

8 Limitations

- **One small model.** Results for SmolLM2-135M need not transfer to larger models, though the middle-redundancy pattern is consistent with what is reported for larger ones.
- **Twelve short prompts, 381 tokens.** Differences of a few points (for example 70.9% against 72.4%) are within what a different prompt set could change. Large gaps (0% against 50% against 90%) are large enough to trust. The prompts are listed in the appendix and are not a representative sample of language.
- **Top-1 agreement is a blunt measure.** It counts a near-tie as a total miss. A softer measure (for example the probability the modified model gives the full model's token) might rank the configurations differently. We did not compute one.
- **No repair.** Dropped layers are removed with no rescaling, retraining or adapter. A repaired model would do better, and this study says nothing about how much.
- **The logit lens is not the model's belief.** Low-depth output reflects what the final output stage can read, not what the earlier stream contains. A learned per-layer correction (a "tuned lens") would read earlier layers better; we did not build one.
- **Greedy and teacher-forced.** We used greedy decoding and scored on the full model's own tokens. Sampling at a temperature would behave differently.
- **Timing is indicative.** One machine, one run per configuration, no repeated measurement in this study.

9 Reproducing the work

The code lives on the branch `experiment/early-exit` of the Emberly Oxygen repository, branched from the 0.4.0 release (commit `c0b8114`), and is not intended for the main line. The experiments are Rust examples in `crates/oxygen/examples/`:

- `early_exit.rs`: Experiments 1 and 2 on one prompt, with free-running text and timing.
- `early_exit_sweep.rs`: Experiment 2 over the twelve prompts, including the one-layer and five-layer sweeps.
- `logit_lens.rs`: Experiment 3.
- `early_exit_samples.rs`: the free-running text in the sample-output appendix.

They are run with `cargo run --release --example <name>` from the repository root. They need the SmolLM2-135M files in `models/smollm2-135m/` (the base model's `config.json`, `model.safetensors` and tokenizer files). Everything is deterministic (greedy decoding), so the numbers should reproduce exactly on the same machine and build; small differences in the last digits of the scores could appear on another machine, which could flip near-ties. The sweep takes about 15 minutes on an Apple M4 Max.

10 Appendix: prompts

1. The quick brown fox jumps over the lazy dog, and then
2. The capital of France is
3. In 1969, the first humans landed on the Moon. The mission was called
4. `def fibonacci(n): / if n < 2: / return n / return` (four lines)

5. Once upon a time, in a small village by the sea, there lived an old fisherman who
6. The three primary colors are red, blue, and
7. Photosynthesis is the process by which plants convert
8. Q: What is 12 plus 30? / A: (two lines)
9. Dear Sir or Madam, / I am writing to complain about (two lines)
10. The mitochondria is the powerhouse of the
11. Water boils at 100 degrees Celsius at sea level, but on a high mountain it
12. 1, 2, 3, 4, 5, 6,

11 Appendix: sample outputs

Percentages hide what a broken model *looks* like, so this appendix shows the actual text. Each model was run with greedy decoding for 40 tokens (it stops earlier if it emits an end-of-text token). Text is shown with Rust's escaping: `\n` is a line break and `\u{a0}` is a non-breaking space. A run of the same token is collapsed to `<filename>×N` where it repeats to the end (that token is a special marker in this model's vocabulary, not text). Configurations are named as in the main text; "drop 3-7" means layers 3 to 7 were removed.

Things to notice:

- The full model is fluent but not reliable (it answers "12 plus 30" with 43). Agreement with the full model is a measure of similarity, not of correctness.
- "First 24 + last 1" produces one wrong token and then the `<filename>` marker forever, on every prompt, including the counting one.
- The counting prompt is easy: even the first 25 layers continue the sequence perfectly. Agreement depends heavily on how hard the next token is.
- Removing layer 11 alone, which looked like a mild loss in the single-layer sweep, produces looping or empty text on most prompts. A top-1 score of 43% corresponds to output that is visibly broken.
- Dropping a block from the middle (15 to 19, 3 to 7, 12 to 16) keeps grammatical text and often the topic, but it loops, repeats itself, or drifts. The "first 15 + last 10" model even writes "Paris" after "The capital of France is", which the full model does not.

Prompt: `The capital of France is`

Configuration	**Output**
all 30 layers	`the capital of the country.\n\nThe capital of France is the capital of the country.\n\nThe capital of France is the capital of the country.\n\nThe capital of France is the`
first 28	`the same as the one in which it is named, and it is a\n C d  D  D  D D  D  D  D  D  D  D  D`
first 25	`\u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0},`
first 24 + last 1	`noqa<filename>×39`
first 15 + last 10 (drop 15-19)	`the capital of the country.\n\nThe capital of France is Paris, the seat of government.\n\nThe capital of France is the capital of the country.\n\nThe capital of France is`
drop 3-7	`the capital of France.\n\nThe capital of France is the capital of France.\n\nThe capital of France is the capital of France.\n\nThe capital of France is the capital of France`
drop 12-16	`the most beautiful of all the arts.\n\nThe most beautiful of the arts is the most of the most of the arts.\n\nThe most beautiful of the arts is the most of the most`
drop layer 11 only	`and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and and`

Prompt: `Once upon a time, in a small village by the sea, there lived an old fisherman who`

Configuration	Output
all 30 layers	`had been fishing for generations. He was known for his wisdom and knowledge of the sea. One day, he met a young girl named Lily who was fascinated by his stories.\n\nLily asked him`
first 28	`was a good- natured and a good- looking man. he was a 100 years old and was a 100 years old. he was a 100 years`
first 25	`was a\n\n. 200 \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0} , \u{a0}, \u{a0}, \u{a0},`
first 24 + last 1	`necessit<filename>×39`
first 15 + last 10 (drop 15-19)	`had a great many children.\n\nThe children were all the way up the river, and the children were all the way up the river, and the children were all the way up the river.`
drop 3-7	`had a great love for the sea. He was a fisherman, and he was a fisherman, and he was a fisherman, and he was a fisherman, and he was a fisherman, and he was`
drop 12-16	`had been fishing for the first time in a long line of fishing boats. The fisherman who had been fishing for the first time in a long line of fishing boats. The fisherman who had been fishing for`
drop layer 11 only	`who who who who who who who who who who who who who who who who who who who who who who who what what what what what what what what what what what what what what what what what`

Prompt:

```
def fibonacci(n):
    if n < 2:
        return n
    return
```

<table>
<tr><th>Configuration</th><th>Output</th></tr>
<tr><td>all 30 layers</td><td><code>fibonacci(n - 1) + fibonacci(n - 2)\n\nprint(fibonacci(10))\n<|endoftext|></code></td></tr>
<tr><td>first 28</td><td><code>1 + 2 * (1 + 2 * (1 + 2 * (1 + 2 * (1 + 2 * (1 + 2</code></td></tr>
<tr><td>first 25</td><td><code>1- (1- (1- (1- (1- (1- (1- (1- (1- (1- (</code></td></tr>
<tr><td>first 24 + last 1</td><td><code>noqa<filename>×39</code></td></tr>
<tr><td>first 15 + last 10 (drop 15-19)</td><td><code>n\n\ndef fibonacci(n):\n   if n == 1:\n   return 1\n   if n == 2 and 1 == 2:\n   return 1\n   if n</code></td></tr>
<tr><td>drop 3-7</td><td><code>fibonacci(n - 2)\n\nfibonacci(2)\n\nfibonacci(3)\n\nfibonacci(4)\n\nfibonacci(5)\n\nfibonacci</code></td></tr>
<tr><td>drop 12-16</td><td><code>n\n   return n\n   return n\n\ndef fib(n):\n   if n == 1:\n return n\n   return n\n\ndef fib(n):\n   return n\n\ndef</code></td></tr>
<tr><td>drop layer 11 only</td><td><code>n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n\n</code></td></tr>
</table>

Prompt: `Q: What is 12 plus 30?`
`A:`

Configuration	Output
all 30 layers	`12 + 30 = 43.\n\nQ: What is 12 plus 30 minus 12?\nA: 12 - 12`
first 28	`12 + 30 is 33.\n\n Q: what is 12 + 30 - 1?\n A: 12 - 1 is`
first 25	`12 \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0}, \u{a0},`
first 24 + last 1	`noqa<filename>×39`

Configuration	Output
first 15 + last 10 (drop 15-19)	` 12 is 12 plus 3, 12, 3, 1, 3, 1, 3, 1, 3, 1,`
drop 3-7	` 120\n\nB: 120\n\nC: 120\n\nD: 120\n\nE: 120\n\nF:`
drop 12-16	` 12 + 30 = 12 + 30 = 12 + 30 = 12 + 30 = 30 + 30 =`
drop layer 11 only	`\n\n\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"\"`

Prompt: `Photosynthesis is the process by which plants convert`

Configuration	Output
all 30 layers	` light energy into chemical energy. The process is a complex one, involving the use of chlorophyll, a green pigment, to capture light energy and convert it into chemical energy.\n\nThe process of photosynthesis`
first 28	` the sun's \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0} \u{a0}`
first 25	`  the  \u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}\u{a0}`
first 24 + last 1	`ptious<filename>×39`
first 15 + last 10 (drop 15-19)	` the sun's energy to food.\n\nThe process of photosynthesis is the process of converting the sun's energy to food.\n\nThe process of photosynthesis is the process of converting the sun's energy`
drop 3-7	` carbon dioxide into oxygen.\n\nThe process is called photosynthesis.\n\nThe process is called photosynthesis.\n\nThe process is called photosynthesis.\n\nThe process is called photosynthesis.\n\nThe`
drop 12-16	` sunlight into energy. The process of photosynthesis is a chemical reaction that occurs in the form of light. The process of photosynthesis is a chemical reaction that occurs in the form of light. The process of photosynthesis`
drop layer 11 only	` to the\n\nof the\n\nof the\n\nof the\n\nof the\n\nof the\n\nof the\n\nof the\n\nof the\n\nof the\n\n`

Prompt: `1, 2, 3, 4, 5, 6,`

Configuration	Output
all 30 layers	` 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17`
first 28	` 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17`
first 25	` 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17`
first 24 + last 1	` noqa<filename>×39`
first 15 + last 10 (drop 15-19)	` 7, 8, 9, 10, 11, 12, 13, 4, 5, 6, 7, 8,`
drop 3-7	` 7, 10, 15, 16, 19, 21, 27, 29, 30, 39,`
drop 12-16	` 7, 8, 9, 10, 10, 10, 10, 10, 10, 10, 10`
drop layer 11 only	` 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,`

12 Appendix: logit-lens listings

Full output of the logit-lens experiment for four prompts. “After” is the number of layers applied (0 is the embedding alone, 30 the real output). The last column is the probability that depth gives to the token the full model chose.

```
=== "The capital of France is" (5 tokens) ===
after  top-3 predictions (probability)                final's top
   30  " the" 26%, " Paris" 9%, " located" 7%             26.2%
   29  " the" 44%, " called" 7%, " D" 4%                  43.5%
   28  " the" 72%, " a" 9%, " " 6%                        72.0%
   27  " the" 49%, " " 29%, " a" 9%                       49.0%
   26  " " 56%, " the" 17%, " a" 8%                       17.1%
   25  " " 39%, " the" 27%, " a" 9%                       27.4%
   24  " " 43%, "," 22%, " the" 14%                       14.4%
   23  " " 64%, "," 14%, " a" 10%                          4.9%
   22  " " 89%, "," 5%, " a" 2%                            0.6%
   21  " " 89%, "," 5%, " a" 2%                            0.2%
   20  " " 97%, "," 2%, " (" 1%                            0.0%
   19  " " 72%, "," 22%, " (" 3%                           0.0%
   18  " " 72%, "," 21%, " (" 4%                           0.0%
   17  " " 58%, "," 30%, " (" 8%                           0.0%
   16  " " 66%, "," 19%, "." 11%                           0.0%
   15  " " 78%, "," 16%, "." 3%                            0.0%
   14  " " 53%, "," 37%, " (" 6%                           0.0%
   13  "," 59%, " " 25%, "." 13%                           0.0%
   12  "," 41%, "." 32%, " " 23%                           0.0%
   11  " " 40%, "." 32%, "," 25%                           0.0%
   10  "." 46%, " " 27%, "," 26%                           0.0%
    9  "." 56%, "," 26%, " " 16%                           0.0%
    8  " " 60%, "," 19%, "." 13%                           0.0%
    7  " " 73%, "." 11%, "," 10%                           0.0%
    6  " " 33%, "," 27%, "." 21%                           0.0%
    5  " " 33%, "," 20%, "." 18%                           0.0%
    4  " (" 29%, "." 25%, "," 19%                          0.0%
    3  "." 54%, " " 17%, "," 12%                           0.0%
    2  "." 70%, " (" 14%, "," 6%                           0.0%
    1  " a" 38%, "\u{a0}" 36%, " so" 8%                    0.1%
    0  " is" 100%, " was" 0%, " are" 0%                    0.0%
```

```
=== "The quick brown fox jumps over the lazy dog, and then" (12 tokens) ===
after  top-3 predictions (probability)                final's top
   30  " the" 19%, " he" 7%, " there" 4%                  18.5%
   29  " the" 24%, " he" 21%, " I" 5%                     24.0%
   28  " he" 36%, " the" 28%, " I" 14%                    27.8%
   27  " the" 34%, " he" 16%, " a" 13%                    34.2%
   26  " a" 29%, " the" 24%, " I" 16%                     23.5%
   25  " the" 31%, " I" 23%, " a" 21%                     30.8%
   24  "," 31%, " a" 22%, " I" 17%                         5.1%
   23  "," 45%, " I" 13%, "\n" 12%                         3.6%
   22  "," 67%, "." 15%, " " 6%                            0.1%
   21  "," 45%, " " 31%, "." 12%                           0.1%
   20  " " 51%, "," 43%, "." 3%                            0.0%
   19  "," 86%, " " 13%, " (" 0%                           0.0%
   18  "," 82%, " " 18%, " I" 0%                           0.0%
   17  "," 95%, " " 4%, "." 0%                             0.0%
   16  "," 89%, " " 10%, " I" 1%                           0.0%
   15  "," 95%, " " 2%, " I" 2%                            0.0%
   14  "," 94%, " I" 3%, " " 1%                            0.0%
   13  "," 89%, " " 5%, "-" 2%                             0.0%
   12  "," 74%, " I" 11%, " " 8%                           0.0%
   11  "," 87%, " " 5%, " I" 4%                            0.0%
   10  "," 85%, " I" 6%, " " 5%                            0.0%
    9  "," 56%, " " 23%, " I" 13%                          0.0%
    8  "," 51%, " " 19%, " I" 17%                          0.0%
    7  "," 55%, " " 29%, " is" 7%                          0.0%
    6  "," 88%, " " 3%, " is" 3%                           0.0%
    5  "," 94%, " " 3%, " is" 1%                           0.0%
    4  "," 67%, " " 14%, " is" 6%                          0.0%
    3  "," 64%, " " 15%, " is" 10%                         0.0%
    2  "," 78%, " is" 14%, " " 4%                          0.0%
    1  " is" 75%, " " 22%, "?" 2%                          0.0%
    0  " then" 100%, "then" 0%, " Then" 0%                 0.0%
```

```
=== "Q: What is 12 plus 30?\nA:" (15 tokens) ===
after  top-3 predictions (probability)                final's top
   30  " " 59%, " It" 5%, " The" 3%                       58.9%
   29  " " 84%, "\u{a0}" 3%, " It" 2%                     83.9%
   28  " " 98%, "\u{a0}" 1%, " one" 0%                    98.4%
   27  " " 98%, "\u{a0}" 1%, " two" 0%                    98.4%
   26  " " 99%, "\u{a0}" 0%, " I" 0%                      99.3%
   25  " " 100%, "\u{a0}" 0%, " I" 0%                     99.6%
   24  " " 100%, "\u{a0}" 0%, " a" 0%                     99.9%
   23  " " 100%, "\u{a0}" 0%, " a" 0%                     99.9%
   22  " " 88%, "\u{a0}" 11%, " I" 0%                     88.5%
   21  " " 74%, "\u{a0}" 18%, " I" 4%                     74.0%
   20  " " 98%, "\u{a0}" 2%, " (" 0%                      97.7%
   19  " " 84%, " (" 13%, "," 1%                          84.4%
   18  " " 85%, " (" 9%, "," 2%                           85.1%
   17  " " 99%, "," 0%, " (" 0%                           98.8%
   16  " " 98%, "," 1%, "." 0%                            98.0%
   15  " " 95%, "," 3%, " (" 1%                           95.2%
   14  " " 92%, "," 6%, "." 1%                            91.8%
   13  " " 68%, "," 20%, "." 5%                           67.7%
   12  " " 66%, "," 15%, "." 10%                          65.9%
   11  " " 89%, "." 4%, " (" 4%                           89.1%
   10  " " 95%, " (" 3%, "," 1%                           95.3%
    9  " " 89%, " (" 6%, "," 3%                           88.9%
    8  " " 89%, "," 5%, " (" 4%                           88.8%
    7  " " 82%, "," 11%, " (" 5%                          82.3%
    6  " " 53%, "," 32%, " (" 9%                          52.8%
    5  " " 41%, "," 38%, " (" 9%                          41.4%
    4  " " 58%, "," 23%, " (" 10%                         57.9%
    3  " " 62%, " (" 8%, " and" 6%                        62.4%
    2  " " 89%, " (" 6%, "," 1%                           88.8%
    1  " " 85%, " no" 3%, " then" 3%                      84.5%
    0  ":" 100%, "," 0%, ";" 0%                            0.0%
```

```
=== "def fibonacci(n):\n    if n < 2:\n        return n\n    return" (18 tokens) ===
after  top-3 predictions (probability)                final's top
   30  " fib" 95%, " n" 1%, " (" 1%                      94.8%
   29  " " 45%, " n" 23%, " N" 2%                           1.8%
   28  " " 89%, " (" 4%, "-" 2%                             0.0%
   27  " " 93%, " (" 3%, "-" 1%                             0.0%
   26  " " 85%, " (" 7%, "-" 4%                             0.0%
   25  " " 89%, " (" 8%, " F" 1%                            0.0%
   24  " " 98%, " (" 1%, "-" 1%                             0.0%
   23  " " 100%, " (" 0%, "-" 0%                            0.0%
   22  " " 99%, "-" 1%, " (" 0%                             0.0%
   21  " " 77%, "-" 20%, " (" 2%                            0.0%
   20  " " 78%, " (" 12%, "-" 7%                            0.0%
   19  " " 40%, "." 37%, " (" 13%                           0.0%
   18  "." 57%, " A" 21%, " (" 5%                           0.0%
   17  "." 87%, " (" 8%, " " 3%                             0.0%
   16  " " 81%, "." 14%, " (" 3%                            0.0%
   15  " " 90%, " (" 5%, "-" 3%                             0.0%
   14  " " 70%, "-" 18%, " (" 8%                            0.0%
   13  "." 41%, "-" 29%, " " 18%                            0.0%
   12  " " 45%, "." 25%, "-" 20%                            0.0%
   11  " " 71%, " (" 14%, "-" 5%                            0.0%
   10  " " 66%, "." 16%, " (" 5%                            0.0%
    9  " " 76%, "." 9%, "," 4%                              0.0%
    8  "-" 33%, "." 22%, " (" 18%                           0.0%
    7  "-" 59%, " " 16%, " (" 12%                           0.0%
    6  "-" 59%, "." 14%, " " 13%                            0.0%
    5  " " 40%, "-" 21%, "." 19%                            0.0%
    4  "." 29%, "-" 26%, " " 12%                            0.0%
    3  "." 54%, "-" 15%, "," 13%                            0.0%
    2  "." 47%, "-" 31%, "," 7%                             0.0%
    1  "ing" 99%, "." 0%, "ed" 0%                           0.0%
    0  " return" 100%, "return" 0%, " returns" 0%          0.0%
```